Nyx cryptography and systems audit - 2026-07-14

> **Scope.** Defensive, first-party review of the devnet-stage Nyx vault, circuits, host cryptography, TEE matcher/settler, SDK key/proving boundary, daemon custody boundary, and settlement performance. Code at `24fbf18` is the ground truth. This report does not assess the website, demo, operational runbooks, or third-party infrastructure.
>
> **Prior-art handling.** Findings already tracked as F-01..F-07 or N-01..N-19 are not re-reported. In particular, the deterministic development Groth16 contribution, external circuit audit, single-signature administration, deliberately TEE-trusted price/limit fairness, and throughput-roadmap items 1-5 remain inherited gates. Where a new issue compounds an existing finding, the distinction is stated explicitly.

Executive summary

The review found one Critical, five High, seven Medium, one Low, and four new performance findings. The highest-priority result is a circuit/on-chain atomicity gap in batch fee notes: the N=16 proof aggregates fees from every private slot into the fee notes attached to slot 0, while the chain neither proves those other slots reference real Merkle leaves nor requires them to settle before slot 0 appends the aggregate notes. A compromised authorized TEE can use fifteen phantom slots to create a proof-backed protocol fee liability without consuming the corresponding inputs.

The next tier is dominated by assumptions that are true in the honest Rust assembler but absent from the circuit or durable state: all slots belonging to one mint pair, output `inner_hash` recoverability, globally unique match IDs, and consistent fee-note identifiers. These are precisely the boundaries where cross-language parity tests can all pass while the protocol invariant is still missing.

No new claim below depends on the known deterministic proving setup weakness. The circuit findings hold even under a sound Groth16 setup and collision- resistant Poseidon/SHA-256.

Severity-ranked backlog

ID	Severity	Category	Finding
CS-01	Critical	Constraints, TEE-trust	Aggregate fee notes can be backed by phantom, never-settled batch slots
CS-02	High	Constraints, TEE-trust	The batch does not bind all slots to one market/mint pair
CS-03	High	Constraints, TEE-trust	Output `inner_hash` values are free witnesses, enabling permanent fund destruction
CS-04	High	Replay, TEE-trust	`match_id` restarts at zero and reuses spend nullifiers
CS-05	High	Other (client custody)	A fixed wallet signature is the complete Nyx master secret

ID	Severity	Category	Finding
CS-06	High	Prover, Other (liveness)	Matcher and prover derive fee notes from different slots
CS-07	Medium	Other (privacy), Tx-budget, CU	`lock_note` publicly discloses the note amount despite not storing it
CS-08	Medium	Replay	Multiple fee batches in one tick reuse the same fee nullifiers
CS-09	Medium	Replay, TEE-trust	Settlement accepts expired input locks
CS-10	Medium	Other (privacy/recovery)	The recovery X25519 key is unsigned and accepts low-order points
CS-11	Medium	Replay	`arrival_nonce` is signed but never enforced
CS-12	Medium	Replay, Other (client custody)	The daemon merge-output counter resets to zero
CS-13	Medium	TEE-trust	Strict daemon attestation fails open when its on-chain key check cannot run
CS-14	Low	Other (cryptographic primitive)	The function named KMAC256 is not NIST KMAC256
P-01	Perf-Nit	Other (account locks)	A read-only batch marker is declared writable in every Tx D
P-02	Perf-Nit	Allocation, Prover	The complete N=16 batch tree is recomputed once per inclusion path
P-03	Perf-Nit	Allocation, Other	Clearing and paging repeatedly clone, sort, and scan the full book
P-04	Perf-Nit	Other (RPC)	Concurrent Tx D confirmation fans out one status poll per transaction

Crypto and soundness findings

CS-01 - Aggregate fee notes can be backed by phantom slots

Severity: Critical
Category: Constraints, TEE-trust

Anchors

- `circuits/templates/match_batch.circom:137–200` reconstructs each slot's notes from private openings but contains no Merkle-membership or active-slot constraint.

- `circuits/templates/match_batch.circom:465-512` sums fees over all N slots and binds the entire sum into slot 0's two fee notes.
- `programs/vault/src/instructions/verify_match_batch.rs:47-60,81-107` creates a marker for the root without recording active slots or settlement progress.
- `programs/vault/src/instructions/tee_forced_settle_batched.rs:445-515` appends slot 0's aggregate fee notes in the same transaction as only slot 0's two input consumptions.
- `programs/vault/src/instructions/close_batch_validity_marker.rs:56-82` permits the payer to close the marker without proving every slot settled.

Failure scenario / regression sketch

1. Configure a non-zero fee rate and protocol owner.
2. Build slot 0 from one real locked bid/ask pair.
3. Fill slots 1..15 with arbitrary private note openings. For example, at 30 bps use `base=quote=1_000_000`, `fee=3_000`, zero change, and input amount `1_003_000`. None of their input commitments needs to exist on-chain.
4. Prove the N=16 circuit. The aggregate slot-0 fee notes include all phantom fees and the proof verifies.
5. Settle only slot 0. Its Tx D consumes only its two real inputs but appends the aggregate fee notes. Close or let the marker expire.
6. The protocol owner can produce a normal VALID_SPEND proof for the inflated fee note and withdraw real SPL from the shared vault, leaving legitimate note liabilities insolvent.

This is distinct from N-12. N-12 described early marker close as a liveness risk; CS-01 shows that incomplete batch settlement is already a solvency risk before the marker is closed.

Recommended fix

Prefer per-match fee notes: bind each slot's base and quote fees to fee notes in that slot and append them in the same Tx D that consumes that slot's inputs. The payload already carries both fee commitments, so this need not grow Tx D, although it increases tree leaves and fee-note fragmentation. Derive each fee note from a globally unique per-match identifier.

If batch aggregation must remain, add an on-chain batch state with a public active bitmap/count, mark each active position exactly once, and mint/append the aggregate fee notes only in a finalization instruction after all active positions have consumed their inputs. Merely proving membership of phantom slots is insufficient; consumption must precede fee issuance.

Lockstep: Yes - circuit, witness/padding, matcher fee flush, TEE assembler, vault fee append/finalization, SDK helpers, zkey/VK, N=16 fixture, and recovery tests.

CS-02 - The batch does not bind all slots to one market/mint pair

Severity: High
Category: Constraints, TEE-trust

Anchors

- `circuits/templates/match_batch.circom:386-411` accepts independent base and quote mint halves for every slot.

- `circuits/templates/match_batch.circom:413-456` instantiates the slots without any cross-slot mint equality.
- `circuits/templates/match_batch.circom:485-512` denominates the aggregate buyer and seller fee notes using only slot 0's quote/base mints.
- `crates/nyx-tee/src/settle/assemble.rs:138-145` checks the configured pair on the honest path, but that check is not part of the proof.

Failure scenario / regression sketch

Prove slot 0 over `(base=X, quote=Y)` and slot 1 over `(base=Y, quote=X)` or an unrelated pair. Each slot conserves its local assets, so the proof passes. The fee sums from slot 1 are nevertheless minted as X/Y fee notes selected from slot 0. Even if every slot settles, liabilities are removed from slot 1's mints and added to slot 0's mints in raw base units. A later slot-0 fee withdrawal can drain a mint whose private-note obligations were never reduced by that amount.

Independently of fees, a compromised TEE can use the omission to settle a victim into a different asset pair, contradicting the documented claim that the proof prevents output mis-minting. This is stronger than the explicitly accepted price-fairness risk.

Recommended fix

At minimum constrain `base_mint[i] == base_mint[0]` and `quote_mint[i] == quote_mint[0]` for every slot, including pads. To prove the intended market rather than merely internal consistency, make the configured base/quote mint halves public inputs bound to authoritative on-chain market configuration. Per-match fee notes from CS-01 also remove the cross-mint fee aggregation failure but do not by themselves bind the intended market.

Lockstep: Yes - circuit public inputs, VaultConfig/market state and verifier, prover witness/padding, zkey/VK, N=16 fixture, and all Rust/TS circuit helpers.

CS-03 - Output `inner_hash` values are free witnesses

Severity: High
Category: Constraints, TEE-trust

Anchors

- `circuits/templates/match_batch.circom:113-123,158-200` accepts `c_inner..f_inner` as unconstrained private values used only in output hashes.
- `circuits/templates/match_batch.circom:307-319` binds only resulting commitments and slot index into the leaf; `match_id` is absent.
- `crates/nyx-tee/src/settle/assemble.rs:190-262` performs deterministic trade, final-change, or anchor derivation only in honest host code.
- `programs/vault/src/instructions/tee_forced_settle_batched.rs:445-515` irreversibly consumes inputs and appends those commitments.
- `packages/sdk/src/fills/recover.ts:59-125` can recover only the documented `derive_inner(match_id, role)` or anchor candidates.

Failure scenario / regression sketch

A compromised authorized TEE chooses arbitrary Fr-safe `c_inner`, `d_inner`, and non-continuation `e_inner/f_inner`, then builds otherwise conservative outputs to the correct owner commitments. The Groth16 proof and settle pass. The users do not know the selected inner values, cannot derive their nullifiers, and cannot produce VALID_SPEND witnesses. Their inputs are permanently consumed and the outputs are permanently unspendable. A live fill memo detects some continuation substitutions only after irreversible settle; it does not prevent arbitrary trade-output inners.

Recommended fix

Use a circuit-friendly, owner-recoverable derivation and enforce it in MatchSlot. A practical design is to derive each output inner via domain- separated Poseidon from the corresponding consumed input inner plus a role and a batch-unique value already bound by the circuit. For continuation notes, turn the anchor mechanism into a constrained deterministic chain or publicly commit the client-supplied anchor root and prove membership. Avoid adding in-circuit SHA-256 solely to preserve the current helper if a Poseidon PRF removes substantial constraints.

Lockstep: Yes - circuit, Rust/TS derivation, anchor design, recovery logic, zkey/VK, fixture, and parity/KAT tests.

CS-04 - `match_id` restarts at zero and reuses spend nullifiers

Severity: High
Category: Replay, TEE-trust

Anchors

- `crates/nyx-tee/src/matcher/interval.rs:58-64,93-108` stores a volatile `next_match_id` initialized to zero.
- `crates/nyx-tee/src/matcher/interval.rs:566-594` advances it only in memory.
- `crates/nyx-tee/src/main.rs:257-261` creates a fresh MatcherState every boot.
- `crates/nyx-tee/src/persistence/snapshot.rs:1-3` is only a persistence stub.
- `crates/darkpool-matcher/src/change_note.rs:88-107` derives output inner from only `(u64 match_id, role)`.
- `packages/sdk/src/fills/recover.ts:59-65,113-117` likewise uses only the low u64 carried in the 16-byte field.

Failure scenario / regression sketch

The same user receives a buyer trade output at match 0, the CVM restarts, and the user receives another buyer trade output at the new match 0. Both notes use the same spending key, inner, and therefore the same amount-independent nullifier. Spending either creates the nullifier PDA and permanently prevents spending the other. Different amounts avoid a commitment collision but do not avoid the nullifier collision.

Recommended fix

Use a globally unique 128-bit settlement identifier, e.g. a rollback-resistant boot epoch plus a monotonic counter, and consume the full identifier in all inner derivations. Durable counter state must have an anti-rollback story; an encrypted disk snapshot alone does not provide monotonicity against volume rollback. CS-03's input-inner-based construction can remove dependence on a global counter for user outputs.

Lockstep: Yes - matcher, payload encoding, TEE assembler, SDK recovery, Rust/TS parity vectors, and potentially the circuit under the CS-03 fix.

CS-05 - A fixed wallet signature is the complete Nyx master secret

Severity: High
Category: Other (client custody)

Anchors

- `packages/sdk/src/keys/key-generators.ts:83-110` maps an Ed25519 signature of the public fixed string `NYX_DARKPOOL_SEED_V1` directly to the master seed.
- `packages/sdk/src/keys/key-generators.ts:112-136` uses it to derive the spending and viewing keys.
- `packages/sdk/src/providers.ts:45-60` exposes this as a normal no-backup mode.

Failure scenario / regression sketch

Any dapp or phishing page asks the wallet to sign the known fixed message. Deterministic Ed25519 signing returns the same signature Nyx uses. The requester hashes that signature, derives the spending key, enumerates/reconstructs notes, and withdraws them. No Nyx origin, session, or application secret is involved.

Recommended fix

Do not make an exportable message signature the spend authority. Generate a random master seed and store it encrypted under a wallet-backed wrapping key, or use a wallet PRF/hardware derivation API whose output is not returned as a portable signature. Origin-specific messages reduce cross-site replay but do not protect against a compromised Nyx frontend and complicate recovery; treat them only as a transitional mitigation. Existing wallet-signature accounts need an explicit migration plan.

Lockstep: Key architecture/migration change, not a circuit change. All key derivation and recovery tests must be versioned.

CS-06 - Matcher and prover derive fee notes from different slots

Severity: High
Category: Prover, Other (liveness)

Anchors

- `crates/darkpool-matcher/src/lib.rs:217-239` passes the matcher's `current_slot` to fee flush and stores it on RunBatchOutput.
- `crates/darkpool-matcher/src/algorithm.rs:610-637` derives fee inners and commitments from that slot.
- `crates/nyx-tee/src/settle/scheduler.rs:326-348` ignores `output.batch_slot`, samples the current atomic slot again, and calls it `fee_slot`.
- `crates/nyx-tee/src/settle/assemble.rs:312-315,476-492` derives witness fee inners from the scheduler slot while copying the matcher's commitments.
- `circuits/templates/match_batch.circom:485-512` requires those two values to reconstruct the same commitments.

Failure scenario / regression sketch

Create a fee-bearing RunBatchOutput at slot `S`, then advance the scheduler's atomic slot to `S+1` before `drive_batch`. The matcher commitment contains `derive_inner(S, fee_role)` but the proof witness contains `derive_inner(S+1, fee_role)`. Witness generation or proving fails for the entire batch. The current unit tests use the same slot and miss the race.

Recommended fix

Carry one explicit fee/batch identifier in RunBatchOutput and use that exact value for both commitment and witness construction. Do not re-sample time at the consumer. Prefer the globally unique identifier from CS-04/CS-08 rather than a slot.

Lockstep: Rust matcher/TEE change. Circuit changes only if the identifier or fee architecture changes under CS-01.

CS-07 - `lock_note` publicly discloses the note amount

Severity: Medium
Category: Other (privacy), Tx-budget, CU

Anchors

- `circuits/valid_input/circuit.circom:57-75,93-115` declares amount public.
- `programs/vault/src/instructions/lock_note.rs:24-34,75-134` carries amount in instruction data and verifier public inputs.
- `programs/vault/src/instructions/lock_note.rs:136-167` explicitly does not store the amount but emits it in NoteLocked.
- `docs/ARCHITECTURE.md:3-7` broadly claims order amount never appears on-chain.

Failure scenario / regression sketch

When a settlement-created trade note is later used as collateral, its exact previously private amount is published in Tx A and linked to its commitment, order ID, mint, and subsequent match graph. For exact-collateral orders this is also the order notional. Initial deposits already expose deposit amount, but lock links that public deposit to hidden order activity; later trade notes leak an amount that was not previously public.

Recommended fix

Make amount a private VALID_INPUT witness, add the already-tracked 64-bit range constraint, and remove it from the lock instruction and NoteLocked event. The commitment plus ownership/membership proof is sufficient; MatchSlot later opens the same commitment and proves value conservation. This also removes 8 instruction/event bytes and one Groth16 public input, improving Tx A headroom and verifier CU.

Lockstep: Yes - circuit/zkey/VK, verifier public input count/order, TEE lock encoder, SDK IDL/prover, event decoders, and transport/roundtrip tests.

CS-08 - Multiple fee batches in one tick reuse fee nullifiers

Severity: Medium
Category: Replay

Anchors

- `crates/nyx-tee/src/matcher/interval.rs:462,508-545` passes the same `now_slot` to every capped page produced in one tick.
- `crates/darkpool-matcher/src/algorithm.rs:610-637` uses only that slot and fee role for both fee inners.
- `CRYPTOGRAPHY.md:394-400` defines the amount-independent nullifier as `Poseidon3(DOMAIN_NULL, spending_key, inner_hash)`.

Failure scenario / regression sketch

Place enough crossing orders to produce two N=16 pages in one matcher tick with non-zero fees. Both quote fee notes share one inner and both base fee notes share another. Even if their amounts and commitments differ, each pair has the same protocol nullifier. Withdrawing one makes the other unspendable. Equal fee totals can additionally create duplicate commitments.

Recommended fix

Derive fee outputs from a unique batch/page identifier, not Solana slot. This should be the same identifier carried consistently under CS-06. Per-match fee notes under CS-01 should use the globally unique match/output identifier.

Lockstep: Rust fee derivation plus protocol-side TS recovery/KATs; circuit lockstep if fee-note construction changes.

CS-09 - Settlement accepts expired input locks

Severity: Medium
Category: Replay, TEE-trust

Anchors

- `programs/vault/src/instructions/lock_note.rs:104-110` bounds lock creation.
- `programs/vault/src/instructions/release_lock.rs:21-24` treats a lock as releasable at `clock.slot >= expiry_slot`.
- `programs/vault/src/instructions/tee_forced_settle_batched.rs:321-325,393-402` loads mint/order ID but never checks either lock expiry.

Failure scenario / regression sketch

Lock a note at `expiry_slot=E`, create a still-valid batch marker, then submit Tx D at slot E before anyone races `release_lock`. Settlement succeeds even though the order's signed time-in-force and lock are expired. The exact boundary is inconsistent: release is valid at E, but settle is also valid at E.

Recommended fix

Cache both lock expiry slots in the existing single loads and require `clock.slot < lock_a.expiry_slot` and `< lock_b.expiry_slot` before proof-marker or state mutation. Add exact-boundary and one-side-expired litesvm tests.

Lockstep: No wire/circuit change.

CS-10 - The recovery X25519 key is unsigned and accepts low-order points

Severity: Medium
Category: Other (privacy/recovery)

Anchors

- `packages/sdk/src/orders/build-order.ts:90-98` marks `viewingPubkey` unsigned.
- `packages/sdk/src/orders/canonical.ts:78-98` omits it from OrderCanonical.
- `crates/nyx-tee/src/api/orders.rs:135-146,348-353` accepts any 32 bytes and describes substitution as self-harm.
- `crates/darkpool-crypto/src/fill_encryption.rs:73-93` performs X25519 without rejecting a non-contributory shared secret.
- `crates/nyx-tee/src/settle/fill_recovery.rs:98-138` writes the resulting ciphertext into the signed settle payload.

Failure scenario / regression sketch

A request-mutating gateway replaces the viewing key while leaving the trading signature valid. A valid attacker key redirects durable change-amount recovery to the attacker and strands the rightful owner's recovery path. An all-zero recipient key produces an all-zero X25519 shared secret (confirmed with the installed implementation), so the AEAD key is computable from public context and the on-chain change amount becomes publicly decryptable.

Recommended fix

Add `viewing_pubkey` to a versioned order canonical body and reject low-order / non-contributory X25519 inputs (`was_contributory()` or equivalent). Add a KAT for all-zero and known low-order encodings plus canonical perturbation tests.

Lockstep: Rust/TS canonical version, OpenAPI/request builders, signature KATs. No circuit change.

CS-11 - `arrival_nonce` is signed but never enforced

Severity: Medium
Category: Replay

Anchors

- `crates/darkpool-matcher/src/order_canonical.rs:108-121` says the monotonic nonce is used to reject submit replay.
- `crates/nyx-tee/src/api/orders.rs:432-449` only hashes and verifies it.
- `crates/nyx-tee/src/api/state.rs:228-235,326-328,671-693` relies instead on a bounded, volatile order-ID idempotency map.
- Repository-wide uses of `arrival_nonce` are limited to serialization/tests; no per-trading-key high-water mark exists.

Failure scenario / regression sketch

Capture a valid order request, let the user cancel it while its signed expiry is still in the future, then restart the CVM or evict its idempotency record after 16,384 other orders. Re-submit the exact signed request

under any valid API account. The TEE re-books the canceled intent without a fresh user signature because neither order ID nor nonce is durably rejected.

Recommended fix

Persist a per-trading-key nonce high-water mark and accept only a strictly higher nonce after handling exact idempotent retry semantics. Define recovery and rollback behavior before relying on local disk persistence. A durable used-order-ID set is an alternative if nonce gaps/reordering are undesirable.

Lockstep: TEE state/persistence and client retry semantics; canonical bytes need not change.

CS-12 - The daemon merge-output counter resets to zero

Severity: Medium
Category: Replay, Other (client custody)

Anchors

- `packages/daemon/src/merge-runner.ts:45-76` defaults an in-memory merge counter to zero.
- `packages/daemon/src/merge-runner.ts:93-105` uses it to build each output.
- `packages/sdk/src/utxo/merge.ts:107-119` derives the output inner from that counter.
- `packages/daemon/src/store.ts:27-56` persists notes/orders but no merge counter.
- `packages/daemon/tests/cvm-daemon-lifecycle.test.ts:339-347` constructs the real runner without an explicit persisted starting index.

Failure scenario / regression sketch

Run one merge at index 0, restart the daemon, and run another. Both outputs use the same owner spending key and inner, hence the same nullifier. Once either is spent, the other merged note is permanently unspendable.

Recommended fix

Allocate and persist the merge index transactionally before submitting the merge, or derive output inner from the consumed commitments so uniqueness does not depend on mutable client state. Handle crash-after-submit-before-store by reserving indices rather than rolling them back.

Lockstep: Daemon store/migration and SDK derivation tests; circuit only if the derivation changes.

CS-13 - Strict daemon attestation fails open on on-chain key-check errors

Severity: Medium
Category: TEE-trust

Anchors

- `packages/daemon/src/daemon.ts:236-268` returns success on RPC errors or a missing VaultConfig.
- `packages/daemon/src/daemon.ts:279-299` marks the daemon started after that return.
- `packages/daemon/src/config.ts:33-43` describes strict attestation and the default-on cross-check, but explicitly documents the fail-open behavior.

Failure scenario / regression sketch

Point the daemon at a genuinely attested but stale CVM whose signer was rotated out of VaultConfig, then make the configured RPC unavailable during startup. DCAP and measurement pins pass; the authoritative key-set comparison is skipped; the daemon sends private orders to an enclave that cannot settle the active vault and may be outside the intended rotation window.

Recommended fix

When strict mode and on-chain checking are enabled, fail closed on RPC/missing config. For availability, cache the last finalized key set with an explicit short expiry and genesis/program binding, or query multiple RPCs. Keep a separate dev-only fail-open switch rather than coupling it to strict mode.

Lockstep: Daemon only; add startup tests for RPC throw, null config, stale cache, and explicit development override.

CS-14 - The function named KMAC256 is not NIST KMAC256

Severity: Low
Category: Other (cryptographic primitive)

Anchors

- `crates/darkpool-crypto/src/keys.rs:219-245` feeds SP 800-185 encodings into raw SHAKE256.
- `packages/sdk/src/keys/key-generators.ts:286-355` mirrors the construction.
- `packages/sdk/tests/keys-parity.test.ts:80-123` checks Rust/TS parity but has no NIST known-answer vector.

Failure scenario / regression sketch

Raw SHAKE256 and cSHAKE256 use different domain-separation suffixes; prefixing the cSHAKE encoding into SHAKE does not turn it into cSHAKE. For key=`0x40` x32, custom=`nyx-vk`, empty data, 64 bytes, the repository construction produced `04231d34...dc77e4`, while the installed standards-conformant KMAC256 produced `bfa222a3...d5d058`. All parity tests still pass because both Nyx ports share the same non-standard function.

No practical key-recovery attack was identified from this distinction alone; the construction is still SHAKE-based and domain separated. The issue is misstated assurance, lack of standard KATs, and interoperability.

Recommended fix

Either rename and specify the current construction as a Nyx-specific SHAKE KDF, or migrate to a vetted cSHAKE/KMAC implementation and pin NIST KATs. Do not silently replace it: viewing keys, blinding factors, anchor inners, and merge inners are derivation-versioned state and old notes would otherwise be lost.

Lockstep: Rust/TS key derivation and a versioned wallet migration. No circuit change unless derived values are changed without preserving old-note recovery.

Performance findings

P-01 - A read-only batch marker is writable in every Tx D

Severity: Perf-Nit
Category: Other (account locks)

Anchors

- `programs/vault/src/instructions/tee_forced_settle_batched.rs:250-266` declares the shared marker mutable.
- `programs/vault/src/instructions/tee_forced_settle_batched.rs:339-382,542-550` only reads it and deliberately leaves it open.
- `crates/nyx-tee/src/settle/settle_batched.rs:97-109` and `packages/sdk/src/settlement/settle-builder.ts:349-367` both encode it writable.
- `crates/nyx-tee/src/settle/worker.rs:624-630` incorrectly states the concurrent Tx Ds share no writable account.

Trigger / cost

Every match in one batch references the same BatchValidityMarker as writable. Solana therefore takes a shared write lock and cannot execute the otherwise sharded Tx Ds in parallel, undermining the K-tree concurrency design even though tree, fee payer, locks, and consumed PDAs are distinct.

Recommended fix

Remove `mut` from the Anchor account and mark it read-only in both builders. Add a test that every full-batch Tx D shares zero writable keys. This does not change wire data or transaction bytes.

Lockstep: Vault Accounts metadata plus Rust and TS builders; no circuit.

P-02 - The complete N=16 tree is recomputed once per inclusion path

Severity: Perf-Nit
Category: Allocation, Prover

Anchors

- `crates/nyx-tee/src/prover/leaf.rs:120-158` clones all leaves and hashes every internal level for one requested path.
- `crates/nyx-tee/src/settle/worker.rs:614-623` calls it once for every match.

Trigger / cost

A full batch performs 16 x 15 = 240 Poseidon internal-node hashes plus repeated Vec allocations to obtain paths from a tree that has only 15 unique internal nodes. The optimal construction hashes 15 nodes once and extracts all paths.

Recommended fix

Build fixed-size levels once (`16 + 8 + 4 + 2 + 1` nodes), return all sixteen four-sibling paths, and use stack arrays where N=16 is fixed. Add equality tests against the current per-index helper before retiring it.

Lockstep: TEE-only optimization; leaf/root parity must remain byte-identical.

P-03 - Clearing and paging repeatedly clone, sort, and scan the full book

Severity: Perf-Nit
Category: Allocation, Other

Anchors

- `crates/nyx-tee/src/matcher/book.rs:210-233` clones the complete book for a snapshot.
- `crates/nyx-tee/src/matcher/interval.rs:508-545` repeats that snapshot for every <=16-match page.
- `crates/darkpool-matcher/src/algorithm.rs:184-218` scans all bids and asks for every candidate price, O(P x N).
- `crates/darkpool-matcher/src/algorithm.rs:693-704` sorts the cloned sides again even though the source snapshot is already price ordered.

Trigger / cost

A large crossing book produces many pages in one tick. Each page clones the remaining orders, partitions and sorts them, then rescans both sides for every distinct price. CPU and allocation cost grow quadratically per page and are repaid across pages, increasing the risk that intake/matching falls behind before proving is the bottleneck.

Recommended fix

Compute demand/supply curves from price-level aggregates and prefix/suffix sums in O(N log N), then reuse the ordered levels and cumulative state across pages within one tick. Preserve FIFO and the current one-fill-per-order semantics with property tests against the existing pure matcher.

Lockstep: Matcher implementation only; output parity tests are essential. This is separate from the deferred adaptive-cadence roadmap item.

P-04 - Concurrent Tx D confirmation fans out one poll per transaction

Severity: Perf-Nit
Category: Other (RPC)

Anchors

- `crates/nyx-tee/src/settle/worker.rs:649-676` spawns one independent send-and-confirm loop per Tx D.
- `crates/nyx-tee/src/settle/submit.rs:93-146` polls `getSignatureStatuses([single_signature])` in each loop.
- `crates/nyx-tee/src/settle/submit.rs:149-195` already contains a batched multi-signature polling helper.

Trigger / cost

At N=16 the worker can issue sixteen status RPCs per backoff interval, in addition to rebroadcast calls, even though Solana RPC accepts all signatures in one request. Under Helius limits this creates avoidable 429 pressure exactly when Tx D is waiting for ALT activation.

Recommended fix

Send all signed transactions, retain per-tx rebroadcast deadlines, and poll all pending signatures in one batched request. Remove confirmed signatures from subsequent polls and rebroadcast only pending transactions whose deadline has elapsed. Preserve per-signature slot/error reporting.

Lockstep: TEE RPC/worker only. This is distinct from the deferred increase to `SETTLE_CONCURRENCY`.

Stale or contradictory protocol text

These should be fixed with the corresponding remediation, but are not separate security findings:

- `CRYPTOGRAPHY.md:451–453` says conservation is still enforced on-chain from lock amounts; NoteLock.amount was removed and the handler explicitly relies only on the circuit.
- `CRYPTOGRAPHY.md:646` describes VALID_MATCH_BATCH as proving a price band, contradicting the accepted TEE-trusted decision at `CRYPTOGRAPHY.md:100–115` and the actual circuit.
- `CRYPTOGRAPHY.md:493–507` describes slot-derived fee recovery and calls VaultConfig.fee_rate_bps vestigial; the verifier binds that value as a public input at `verify_match_batch.rs:81–102`.
- `docs/ARCHITECTURE.md:3–7` says order amount never appears on-chain without acknowledging the public LockNote amount/event.
- `programs/vault/src/instructions/tee_forced_settle_batched.rs:307–319` says the leaf needs lock mints, but `compute_match_leaf` at lines 92-124 hashes only commitments and batch slot.
- `crates/darkpool-matcher/src/match_result.rs:1–15,28–30` and `crates/darkpool-matcher/src/fee.rs:1–13` still describe the deleted `matching_engine`, on-chain `submit_order`, and old adapter state.

Validation performed

- `cargo test -p darkpool-matcher`: 46 tests passed across unit, parity, and change-inner suites.
- `cargo test -p nyx-tee --lib settle::assemble`: 17 targeted tests passed.
- SDK vitest: `keys-parity`, `settle-builder-batched`, and `valid-input-prover`: 30 tests passed.
- Direct KMAC comparison against installed `@noble/hashes` confirmed the Nyx output differs from standards-conformant KMAC256.
- Direct X25519 test confirmed an all-zero recipient encoding yields an all-zero shared secret in the installed implementation.

The passing suites are important evidence: CS-01 through CS-06 and CS-08 are missing-invariant or cross-stage tests, not simple Rust/TS byte drift already caught by the current parity suite.

What I could not rule out / needs the team

1. **Trusted setup provenance.** I did not validate ceremony transcripts, toxic-waste destruction, or the committed zkey's provenance. N-18 and the external circuit audit remain hard mainnet gates.

2. **Live TDX/Phala control plane.** No live quote, compose hash, governance allowlist, signer rotation, or CVM image was verified in this static pass.
3. **Rollback guarantees.** The intended LUKS/dstack volume rollback model is not implemented in `persistence/snapshot.rs`; the team must define how monotonic IDs/nonces survive restore and volume rollback.
4. **Production key storage.** Browser wallet-adapter behavior, daemon keystore hardening, backups, memory zeroization, and host compromise were not dynamically assessed.
5. **Trade-output discovery.** The SDK has deterministic change recovery, but I did not find an equally explicit durable reconstruction workflow for every `note_c/note_d` trade output. The team should demonstrate cold-device recovery after missing both live streams.
6. **Economic parameter bounds.** Mint decimals, supported market registry, fixed-point price scaling, and behavior if multi-market batching is planned need a written invariant before fixing CS-02.
7. **Full CU/byte benchmarks.** No fresh SBF build, litesvm CU trace, or serialized production Tx A/Tx D size measurement was run. CS-07 and P-01 have clear directional savings, but exact headroom should be remeasured after changes.
8. **Third-party primitive review.** Poseidon parameter security, arkworks, rapidsnark/icicle, Solana's Groth16 syscall, dcap-qvl, and dependency supply chain were treated as external assumptions rather than independently cryptanalyzed.
9. **Accepted fairness boundary.** The absence of trader-limit, uniform-price, and oracle-band constraints is documented as F-11/TEE-trusted and therefore not re-reported. CS-02 shows the same trust boundary currently extends to asset identity; the team must decide whether that extension is acceptable.

Suggested remediation order

1. Disable fee-bearing settlement or set the fee rate to zero until CS-01 is fixed and regression-proved.
2. Design CS-01/CS-02/CS-03 together as one circuit version; regenerate all artifacts and the N=16 fixture in the required lockstep cycle.
3. Fix CS-06 immediately for honest-path liveness, then replace slot-derived identifiers under CS-04/CS-08 before re-enabling fees.
4. Remove wallet-signature seed mode or gate it behind an explicit migration warning before any real-value deployment.
5. Apply the narrow non-circuit fixes CS-09, CS-10, CS-11, CS-12, CS-13, and P-01 while the circuit redesign is underway.
6. Make VALID_INPUT amount private under CS-07 during the next circuit ceremony so the artifact churn is paid once.